

Ferramentas para Testar Backend (Insomnia e/ou Postman)

Objetivo da Aula

Conhecer ferramentas de teste para o ambiente web e utilizá-las para aferir as funcionalidades de Web Services RESTful.

Apresentação

Com o aumento da complexidade dos sistemas, a área de teste de software se expandiu e diversas ferramentas e metodologias surgiram. A estratégia de testes passou a ser dividida em três níveis, que são os testes unitários, testes de integração e testes ponta a ponta – cada um com ferramentais específicos.

Veremos como os testes unitários e de integração podem ser efetuados no NodeJS, tendo como ferramentas o Jest e o Supertest, permitindo um processo de desenvolvimento que busca garantir a qualidade de software.

Já com o sistema completo, veremos como efetuar os testes ponta a ponta, através de ferramentas externas, sendo aqui abordados os produtos Postman, Insomnia e Boomerang – três bons exemplos de ferramentas que permitem testar as requisições no protocolo HTTP.

1. Rotina de Testes do Backend

Uma boa rotina de testes é um elemento essencial para garantir a qualidade do software. Um sistema testado adequadamente permite entregar um produto mais alinhado com as necessidades do cliente, além de exigir menor nível de manutenção após sua implantação.

Existem três grupos principais de testes:

- **Unitários**, que são voltados para funcionalidades pontuais do sistema, na menor fração de código executável, e verificam métodos e funções;

- **Integração**, que verifica o funcionamento dos componentes do sistema de forma integrada, para cumprir operações complexas;
- **Ponta a ponta**, que simula a utilização do sistema pelo usuário, dentro de um ambiente controlado, buscando cenários de aplicação real.

Duas ferramentas de teste muito interessantes para o ambiente NodeJS são o **Jest** e **Supertest**. Jest é um framework de testes em JavaScript que oferece um modelo de utilização extremamente simples. Já o Supertest é uma biblioteca que facilita o teste de requisições HTTP.



Destaque

Para que possamos efetuar testes mais consistentes, daremos continuidade ao projeto de Web Service RESTful da aula anterior.

Abra o projeto do Web Service de Notas e execute o comando apresentado a seguir no terminal interno do VS Code.

```
npm install --save-dev jest
```

Em seguida, no arquivo **package.json**, configure a ferramenta de teste na divisão **scripts**, como pode ser observado a seguir.

```
"scripts": {
  "test": "jest"
},
```

Os arquivos de teste devem ter a extensão **test.js**, e, para verificar como o processo ocorre, crie o arquivo **exemplo1.test.js**, com o código seguinte.

```
function sum(a, b) {
  return a + b;
}
```

```
test('adds 1 + 2 igual a 3', () => {
  expect(sum(1, 2)).toBe(3);
});
```

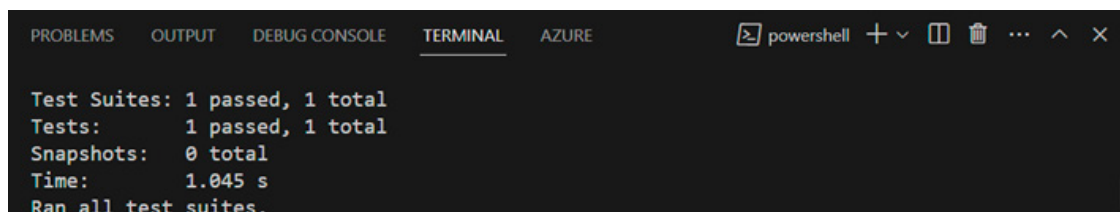
Na listagem, temos a definição de uma função simples de soma e um teste que será efetuado sobre ela. A função **test** define um teste unitário, tendo como parâmetros a descrição do teste e uma função com verificações que farão parte do teste. No exemplo, foi feita uma verificação de igualdade, em que **expect** recebe a chamada da função testada e **toBe** o resultado esperado.

Para executar os testes do projeto, utilize o comando seguinte.

```
npm test
```

Sempre temos a execução de todos os testes unitários do projeto, e não de um subgrupo, e por isso costumamos chamar o conjunto de “suíte” de testes. Para o nosso exemplo, teremos a saída apresentada a seguir.

Figura 1: Execução de testes com Jest



```
Test Suites: 1 passed, 1 total
Tests:      1 passed, 1 total
Snapshots:  0 total
Time:       1.045 s
Ran all test suites.
```

Fonte: Elaboração própria.

O framework Jest não exige configurações complexas, funcionando de forma direta com tecnologias como Babel, JavaScript, Type Script e outras. Os testes são executados **isoladamente**, colocando cada teste em seu próprio processo, com execução em paralelo, para melhorar o desempenho.

Ele também oferece **snapshots**, que são resultados de execuções anteriores armazenados, os quais podem ser comparados com novas execuções. Outra ferramenta disponível no Jest são os **mock servers**, ou servidores simulados, que visam substituir servidores reais para a execução de testes.

Diversos operadores de teste são oferecidos no Jest. Alguns deles podem ser observados no quadro seguinte.

Quadro 1: Alguns dos operadores oferecidos pelo Jest

Operador	Descrição	Exemplo
toBe	Verifica se o valor é igual.	<code>expect(sum(1,2)).toBe(3)</code>
toHaveLength	Verifica o tamanho da coleção.	<code>expect([1, 2, 3]).toHaveLength(3)</code>
toBeDefined	Verifica se não é undefined.	<code>expect(funcaoX()).toBeDefined()</code>
toBeInstanceOf	Testa se é do tipo da classe.	<code>expect(new A()).toBeInstanceOf(A)</code>
toBeNull	Verifica se é nulo.	<code>expect(f1()).toBeNull()</code>
toContain	Testa se faz parte da coleção.	<code>expect([10, 20, 30]).toContain(20)</code>

Fonte: Elaboração própria.

Muitas outras funcionalidades são oferecidas pelo Jest, como a especificação de processos que devem ser iniciados antes de todos os testes (**beforeAll**) ou antes de cada teste (**beforeEach**), bem como de ações que devem ocorrer após os testes (**afterAll** e **afterEach**). Apesar da grande diversidade de funcionalidades oferecidas, o Jest tem uma documentação muito completa, cobrindo o espectro completo de utilização, com muitos exemplos disponíveis.

Para efetuar os testes de nossa API, precisaremos de uma ferramenta que seja voltada para testes de **integração**, e aqui utilizaremos o **Supertest**. Antes de começar, precisamos preparar uma base de dados e devemos configurar a conexão do banco de **teste**, no arquivo **config.json**, como apresentado a seguir.

```
"test": {
  "username": "root", "password": "admin123",
  "database": "agenda_test", "host": "127.0.0.1",
  "dialect": "mysql"
},
```

Quando realizamos testes de integração, é normal criar uma base de dados separada, para que os dados de teste não tragam funcionalidades errôneas para a versão de desenvolvimento. Após configurar a conexão, execute os comandos que são apresentados a seguir.

```
npx sequelize-cli db:create --env test  
npx sequelize-cli db:migrate --env test
```

O primeiro comando cria o banco de dados **agenda_test** no MySQL, e, com a execução dos **migrations**, serão geradas as tabelas necessárias. Note que os comandos especificam o ambiente através do atributo **env**.

Agora, vamos instalar a biblioteca **Supertest**, utilizando o comando seguinte.

```
npm install supertest
```

Para que seja possível testar o servidor, ele deve ser exportado, através da cláusula **exports** de module, na última linha de **index.js**.

```
// O restante do código não deve ser alterado  
module.exports = app
```

Agora, vamos criar o arquivo **index.test.js**, com o código seguinte.

```
const server = require('./index.js')  
const request = require('supertest');  
describe('Teste do servidor', () => {  
  it('Criacao de nota', async () => {  
    const response = await request(server)  
      .post('/notas').send({  
        titulo: "Tarefa Teste",  
        descricao: "Apenas um Teste"  
      });  
    expect(response.status).toBe(200);  
  });  
});
```

```
expect(response.body.titulo).toBe("Tarefa Teste");
});
});
```

No corpo do teste, uma **requisição** via Supertest é feita ao nosso servidor, em modo **post**, com o envio de uma nota no formato JSON. Como o retorno de uma chamada POST, retorna o objeto original acrescido do id, e podemos testar se a informação enviada retornou idêntica e se o código de resposta é **200**.

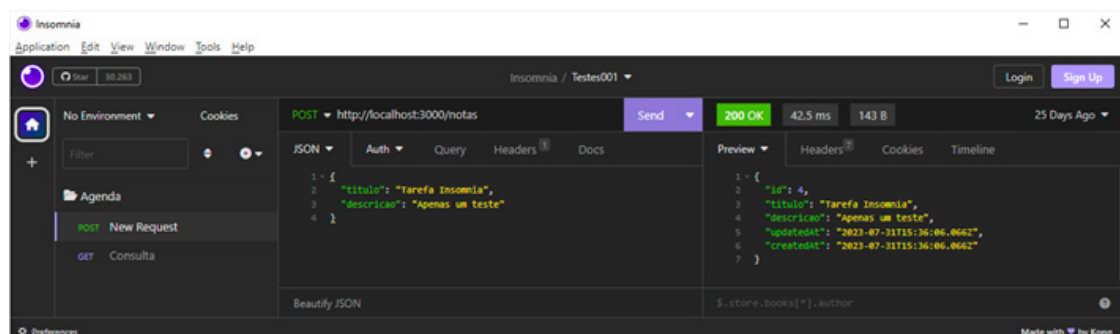
A execução dos testes é iniciada da mesma forma que antes, via comando **npm**, e podemos testar todas as outras chamadas que foram programadas em nosso servidor com base na mesma técnica.

Os testes de **ponta a ponta** devem ser feitos por meio de ferramentas como Insomnia, Postman ou Boomerang, pois simulam o uso da API por alguma outra plataforma de desenvolvimento. Elas serão explicadas nos tópicos seguintes.

2. Ferramenta Insomnia

Insomnia é um aplicativo desktop open source que permite projetar e testar APIs com grande facilidade. Em termos de **projeto**, ela utiliza a especificação **Open API** de documentação, permitindo o projeto e a depuração da API antes de seu desenvolvimento. Em termos de **testes**, oferece amplo suporte aos diversos formatos de dados mais comuns, com todos os métodos do protocolo **HTTP**, além de aceitar diferentes formas de **autenticação**.

Figura 2: Aplicativo Insomnia em execução



Fonte: Elaboração própria.

É possível utilizar o aplicativo de forma individual ou em um **time**, mas, para usar o modelo colaborativo, é necessário pagar uma assinatura, pois, atualmente, o **versionamento** é feito pelo **Insomnia Cloud**. Com a utilização de times, todas as configurações do ambiente de testes podem ser compartilhadas rapidamente com os membros de cada time.

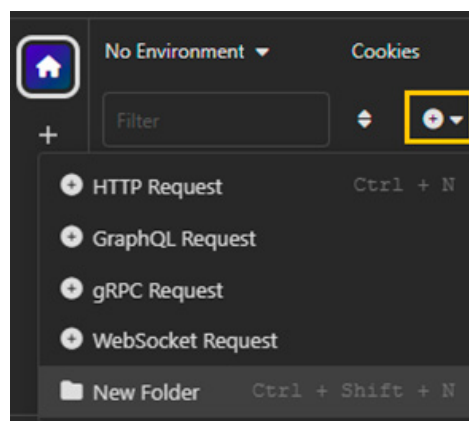
Para instalar o Insomnia, você pode fazer o download ou utilizar o comando apresentado a seguir. Nele é usado o Chocolatey, um gerenciador de instalações para Windows, com diversas funcionalidades avançadas.

```
choco install insomnia-rest-api-client
```

Embora o Insomnia seja muito poderoso, utilizaremos ele apenas para testar as requisições de nosso Web Service de Notas. Em termos de organização, é possível definir ambientes (**Environment**) constituídos de pastas (**Folder**), nas quais ficam armazenadas as requisições que serão testadas.

Para organizarmos nossos testes, vamos criar a pasta Agenda, utilizando o clique sobre o botão de adição circular, no painel esquerdo, e escolhendo a opção **New Folder**, como pode ser observado na figura seguinte.

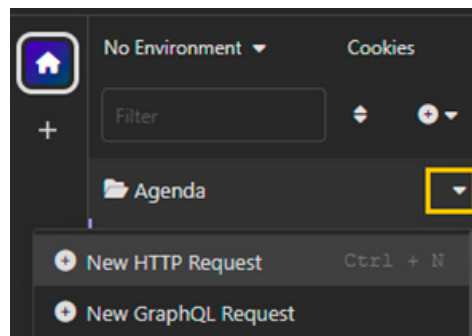
Figura 3: Criação de pasta no Insomnia



Fonte: Elaboração própria.

Além de requisições **HTTP**, de nosso interesse, o aplicativo oferece suporte aos protocolos **GraphQL**, **gRPC** e **Web Socket**. Vamos criar uma requisição na pasta **Agenda**, clicando no botão auto-ocultável, com uma seta apontando para baixo, e escolhendo a opção **New HTTP Request**, como na figura seguinte.

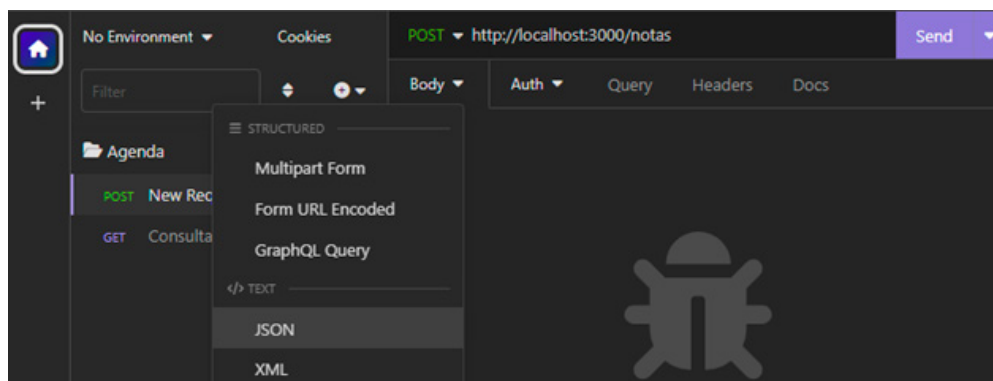
Figura 4: Criação de requisição HTTP na pasta



Fonte: Elaboração própria.

Utilize o endereço **http://localhost:3000/notas**, mude o método HTTP da requisição para **POST** e, na aba **Body**, escolha a opção **JSON**. Em seguida, digite os dados para uma nova nota.

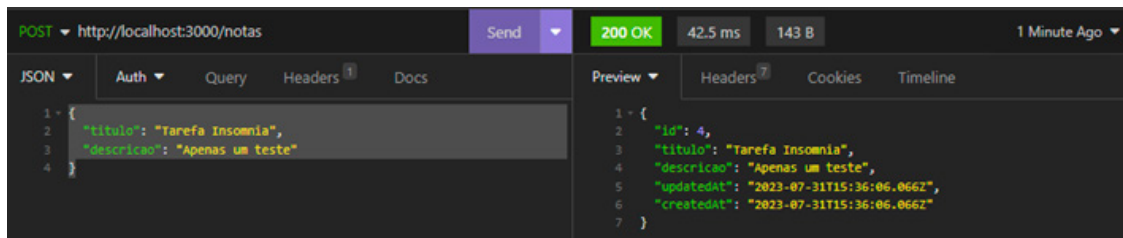
Figura 5: Escolha do formato para o corpo da requisição



Fonte: Elaboração própria.

Com o Web Service em execução, clique no botão **Send**, e você poderá ver o retorno no formato JSON e o código de resposta 200. Temos, ainda, algumas informações de desempenho, como tempo de resposta e quantidade de bytes, além dos Headers da resposta e da visualização de cookies.

Figura 6: Visualização da resposta do servidor no Insomnia



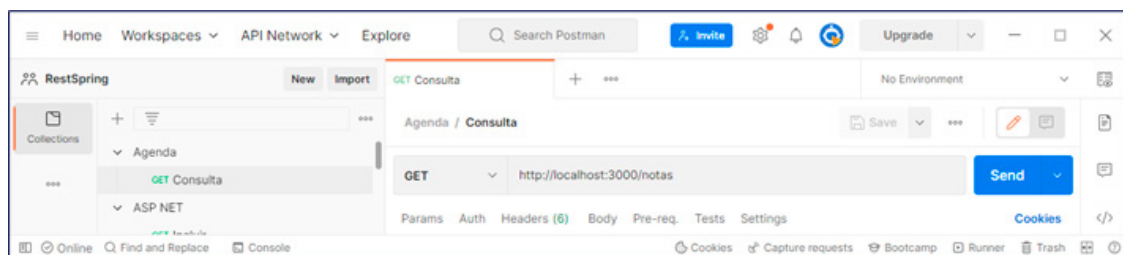
Fonte: Elaboração própria.

Como podemos observar, a verificação de nosso servidor pode ser feita de forma simples com o Insomnia. Todos os outros serviços oferecidos podem ser testados, segundo um processo que se mostrou bastante intuitivo.

3. Ferramenta Postman

Postman é uma ferramenta similar ao Insomnia, com os mesmos recursos e uma organização levemente diferente, em que o **Workspace** agrupa **Collections**, e nelas são adicionadas as requisições. Ele também inclui funcionalidades avançadas, como a criação de **mock servers** e monitores de desempenho e a definição de fluxos de execução, com orquestração de serviços.

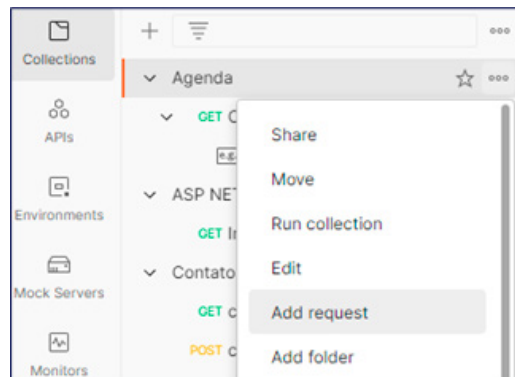
Figura 7: Aplicativo Postman em execução



Fonte: Elaboração própria.

Após definir um Workspace e incluir uma Collection, acrescente a requisição que deseja testar, preenchendo o **endereço** e escolhendo o **método HTTP**.

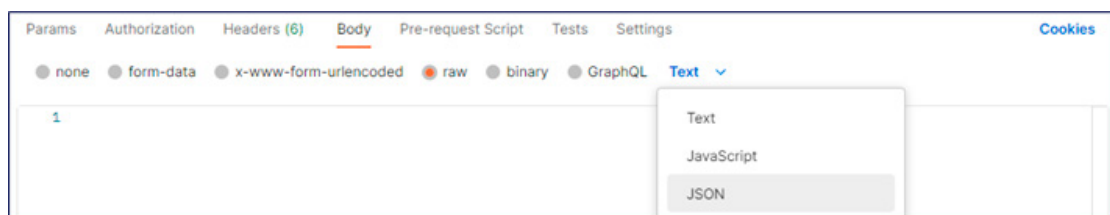
Figura 8: Criação de requisição no Postman



Fonte: Elaboração própria.

A forma de utilização é a mesma do Insomnia, mas, na aba **Body**, devem ser escolhidos **raw** e, depois, **JSON**. Após configurar a requisição, é só clicar em **Send**.

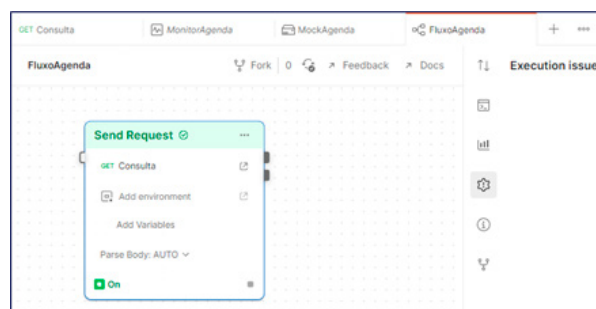
Figura 9: Configuração do corpo da requisição no Postman



Fonte: Elaboração própria.

Uma ferramenta muito interessante é o controle de fluxo, por meio do qual é possível a orquestração de serviços, combinando entradas e saídas via programação por blocos. Para que seja possível utilizar essa ferramenta, é necessário salvar a execução da requisição antes, mas o Postman é muito informativo e indica o que deve ser feito caso falte alguma configuração.

Figura 10: Ferramenta Flows do Postman



Fonte: Elaboração própria.



Para Praticar

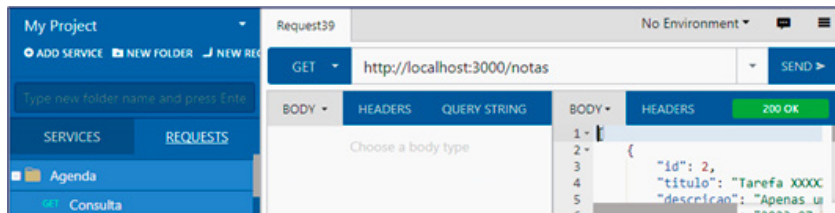
Efetue todas as chamadas disponibilizadas pelo Web Service de Notas tanto no Postman quanto no Insomnia.

4. Plugin Boomerang

A última ferramenta analisada aqui é o **Boomerang**. Diferentemente dos anteriores, Boomerang é um plugin do Chrome que pode ser adicionado mesmo que você não seja administrador do sistema, o que facilita a execução de testes, em redes de ambientes externos, quando não podemos instalar aplicativos.

Sua organização utiliza **Environment** e **Folders**, com as requisições sendo adicionadas no Folder. Podemos criar as requisições sem um Folder, mas, com diversas requisições, sua utilização pode se tornar complicada.

Figura 11: Requisição com método GET no Boomerang

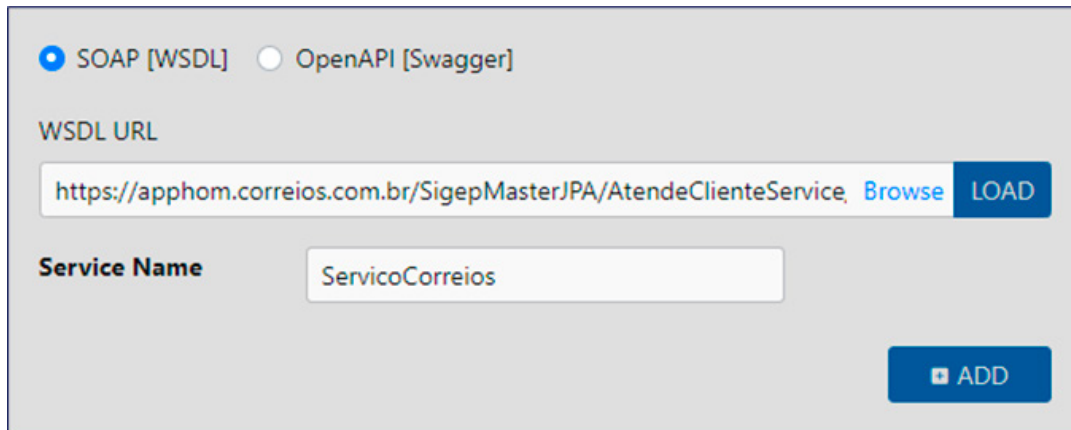


Fonte: Elaboração própria.

Com uma utilização intuitiva, segundo os mesmos moldes do **Insomnia** e do **Postman**, torna-se desnecessário descrevê-la aqui. Por ser um plugin, não temos funcionalidades avançadas, como projeto de APIs ou controle de fluxo.

No entanto, algo muito interessante no Boomerang é a criação de clientes para Web Services SOAP, bastando fornecer o WSDL. Para experimentar essa funcionalidade, utilize a aba **Services** e clique na opção **Add Service**.

Figura 12: Configuração de serviço SOAP no Boomerang



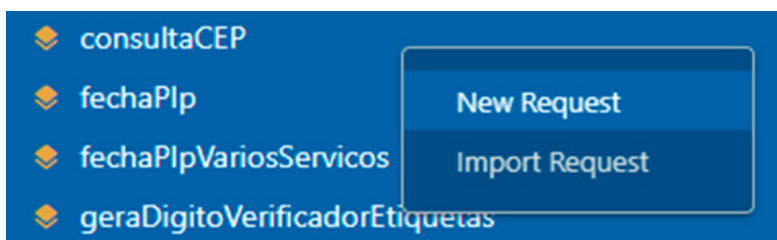
Fonte: Elaboração própria.

Após fornecer o **endereço** da **WSDL** e clicar em **Load**, será solicitado o nome que dará para o serviço no ambiente e bastará clicar em **Add** para criar o cliente de comunicação no Boomerang. Utilize o endereço do serviço para consulta de CEP dos Correios.

`https://apphom.correios.com.br/SigepMasterJPA/AtendeClienteService/AtendeCliente?wsdl`

Com o cliente gerado, todas as operações disponibilizadas ficarão visíveis, logo abaixo dele, no painel de visualização. Procure por **consultaCEP**, clique com o botão direito sobre a operação e escolha **New Request**.

Figura 13: Criação de requisição SOAP no Boomerang



Fonte: Elaboração própria.

Será gerada uma requisição **XML**, em que os parâmetros de consulta ficam com ponto de interrogação. Em nosso exemplo, temos apenas que preencher o **CEP** e devemos **desmarcar** todos os itens da aba Headers, pois esse Web Service, especificamente, não os aceita.

```
<x:Body>  
<cli:consultaCEP>  
<cep>20550200</cep>  
</cli:consultaCEP>  
</x:Body>
```

Agora, é só clicar em **Send**, com método **POST**, e ver os dados do endereço.

Considerações Finais da Aula

O uso de ferramentas de testes é essencial para a garantia da qualidade de software e, por esse motivo, tornou-se uma necessidade primária para diversas metodologias de desenvolvimento, como Scrum e XP. Saber como implementar os testes é essencial para qualquer desenvolvedor, sendo necessário, para o desenvolvedor que trabalha com NodeJS, conhecer ferramentas como Jest, para a criação de testes unitários, e Supertest, utilizado no nível de integração.

Para os testes de ponta a ponta, ferramentas externas são necessárias, e aqui pudemos conhecer o Postman, o Insomnia e o Boomerang. Embora cada uma dessas ferramentas tenha características próprias, o uso de requisições HTTP é bastante similar, permitindo que qualquer uma delas possa ser usada para efetuar os testes de uma API REST.

Aqui, pudemos compreender como utilizar essas ferramentas para testar um Web Service RESTful criado na plataforma NodeJS e vimos, também, como o plugin Boomerang pode ser útil no teste de Web Services SOAP, precisando apenas do endereço que fornece o descritor de serviços no formato WSDL.

Materiais Complementares



Insomnia REST

2023, Insomnia.

O site oficial do Insomnia oferece, além do download, uma boa documentação do produto, com diversos exemplos de utilização.

Link para acesso: <https://insomnia.rest/> (acesso em: 13 set. 2023).



Postman API Platform

2023, Postman.

O site oficial do Postman oferece, além do download, uma boa documentação do produto, com diversos exemplos de utilização.

Link para acesso: <https://www.postman.com/> (acesso em: 13 set. 2023).



Testes Automatizados de APIs com JEST & SUPERTEST

2020, Sofia Trindade.

O artigo descreve a implementação de testes unitários e de integração na plataforma NodeJS, utilizando Jest e Supertest.

Link para acesso: <https://medium.com/beelabacademy/testes-automatizados-de-apis-com-jest-supertest-8aa6a96f61d1> (acesso em: 13 set. 2023).

Referências

FLANAGAN, D. **JavaScript: o guia definitivo**. Porto Alegre: Bookman, 2014. Disponível em: <https://abre.ai/gKcL>. Acesso em: 12 set. 2023.

OLIVEIRA, C; ZANETTI, H. **JavaScript descomplicado: programação para a Web, IoT e dispositivos móveis**. São Paulo: Érica, 2020. Disponível em: <https://abre.ai/gKcN>. Acesso em: 12 set. 2023.

OLIVEIRA, C; ZANETTI, H. **Node.js: programe de forma rápida e prática**. São Paulo: Expressa, 2021. Disponível em: <https://abre.ai/gKcO>. Acesso em: 12 set. 2023.